UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

A LAB EXERCISE 1 REPORT ENTITLED

BUILDING A DISTRIDUTED IDS FOR MULTI-SEGMENT NETWORK MONITORING

BY

NAME: BOTUO THERESAH

INDEX NUMBER: FCM.41.018.109.23

BLUE TEAM

https://github.com/botuothess/CY376_SemesterProject.git

COURSE: NETWORK SECURITY, AUDITING AND MONITORING (CY376)

CLASS: CY 3B

SEMESTER: TWO

COURSE INSTRUCTOR

FREDERICK EDEM BRONI (MR)

TARKWA, GHANA

AUGUST 2026

Abstract

Conventional network intrusion detection deploys a single sensor at the network core and inspects aggregate traffic, an approach that performs adequately on flat networks but breaks down once a network is segmented into zones such as a demilitarized zone (DMZ), an internal corporate LAN, and an operational-technology or Internet-of-Things (OT/IoT) enclave. A central sensor of this kind either cannot observe traffic inside isolated segments at all, or observes it without the per-segment context needed to localize an attacker. This report presents the design, implementation, and evaluation of a distributed intrusion detection system (IDS) that addresses this gap through a lightweight sensor agent deployed on every monitored segment and a central collector that aggregates and correlates the resulting alerts. Each agent runs an identical set of explainable, threshold-based detectors — port-scan detection, SYN-flood detection, ARP-spoof detection, and signature-based payload matching — over traffic that is either captured live or synthetically generated in a simulation mode. A correlation engine running inside the collector groups alerts by source IP address and flags any source whose activity spans two or more segments within a configurable time window, surfacing attack chains such as reconnaissance followed by lateral movement that are invisible to any single sensor. The system was evaluated end-to-end across three simulated segments (DMZ, INTERNAL, and OT_IOT); over one test session it recorded 145 alerts and correctly generated three cross-segment correlated findings, each of which identified a single simulated attacker source spanning multiple segments within the correlation window. The results demonstrate that a distributed architecture with centralized correlation can recover attack patterns that per-segment monitoring alone would miss, while the report also documents the simplifications made for a lab-scale deployment and the steps that would be required to move the system toward production readiness.

Keywords: intrusion detection, distributed systems, network segmentation, alert correlation, lateral movement, cybersecurity monitoring

Table of Contents

1. Introduction

Traditional intrusion detection typically relies on a single sensor positioned at the network core, watching aggregate traffic. This approach works reasonably well for flat, unsegmented networks, but modern networks are rarely flat: a DMZ, an internal corporate LAN, and an OT/IoT zone commonly operate as distinct, sometimes isolated, segments. A single central sensor either cannot see into isolated segments at all, or loses the per-segment context needed to reason about where an attacker actually is.

This project addresses that gap by building a distributed intrusion detection system (IDS): a lightweight sensor agent runs on every monitored segment, and a central collector aggregates and correlates what those sensors see. The result is a system that preserves visibility inside isolated zones while also detecting attack patterns — such as reconnaissance followed by lateral movement — that only become visible when alerts from multiple segments are considered together.

1.1 Objectives

- Design and implement a distributed IDS architecture with per-segment sensor agents and a central collector.
- Implement a set of explainable, rule-based detection techniques covering common network attack patterns.
- Implement a cross-segment correlation engine that identifies attacker behavior spanning multiple segments.
- Provide a live, human-readable dashboard for monitoring alerts and correlated findings in real time.

- Make the system easy to run and demonstrate, with both a simulation mode (no privileged access required) and a live packet-capture mode.

2. Background and Related Work

Interest in distributing intrusion detection across multiple monitoring points dates back to some of the earliest network IDS research. Snapp et al. proposed one of the first systems for distributed intrusion detection, establishing the basic pattern of independent monitors reporting to a central analysis point that this project follows [1]. Treaster's survey of distributed intrusion detection approaches frames the central motivation precisely: aggregating data from distributed sources allows patterns to be seen that would not be detectable if each source were examined in isolation, because a single host or segment rarely has enough evidence on its own to draw a reliable conclusion [2]. More recent surveys confirm that this remains an open problem rather than a solved one — coordinating message exchange between sensors, establishing trust between components, and reaching a joint decision are all still active areas of discussion in the literature on distributed and collaborative intrusion detection networks [3].

A distributed architecture is the standard response to segmented networks for two main reasons. First, it preserves visibility inside segments that are isolated or air-gapped from the rest of the network. An OT or IoT zone, for example, often cannot forward its traffic to a central monitor without breaking the isolation that makes it secure in the first place; a sensor running inside the segment itself does not have this problem. Second, and more importantly for this project, a distributed architecture with centralized correlation can detect attack chains that unfold across multiple segments. An attacker who scans the DMZ and later performs reconnaissance inside the internal network is invisible to any single sensor, because each sensor only ever observes one half

of the story. Zhou et al.'s survey of coordinated attacks makes the same point from the attacker's side: multi-stage, multi-source attacks are specifically designed to stay below the detection threshold of any individual monitoring point, which is precisely what makes cross-source correlation necessary rather than optional [3]. Only a system that aggregates alerts from every segment and looks for the same source appearing in more than one of them can surface this kind of lateral movement. This is the central idea the rest of this report builds on.

At the level of individual sensors, the detection techniques implemented in this project draw on a long-established lineage of signature- and threshold-based network monitoring. Roesch's Snort demonstrated that a lightweight, rule-driven detection engine could achieve broad coverage of common attack signatures without the computational overhead of more complex statistical approaches [4], while Paxson's Bro (now Zeek) showed that a policy-driven event engine could support richer, protocol-aware analysis while remaining explainable to an operator [5]. This project follows the same explainability-first philosophy at the level of an individual sensor, while addressing the segment-visibility and cross-segment-correlation problem that single-node systems such as Snort and Bro do not attempt to solve on their own.

3. System Architecture

The system consists of four cooperating components, shown in Figure 1: per-segment sensor agents, a central collector, a correlation engine, and a live dashboard.

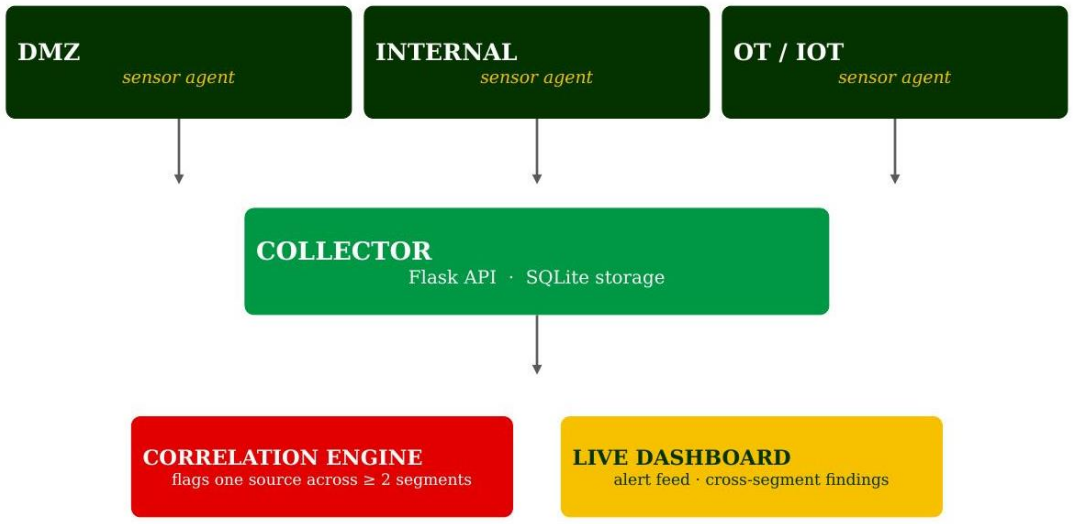

Figure 1. System architecture — per-segment sensor agents report to a central collector, which runs the correlation engine and serves the live dashboard.

3.1 Sensor Agent

One sensor agent runs per monitored segment. Each agent normalizes traffic into flow events and passes them through a shared detection engine. Agents can run in two modes: a simulation mode that generates realistic synthetic traffic (including periodically injected attacks) so the system can be demonstrated without special privileges, and a live mode that captures real traffic from a network interface or mirror port using scapy.

3.2 Collector

The collector is a Flask application exposing a small REST API. Sensor agents authenticate with a shared API key and POST alerts to it; the collector persists every alert to a SQLite database and exposes read endpoints for the dashboard.

3.3 Correlation Engine

The correlation engine runs as a background thread inside the collector process. On a fixed interval it scans recent alerts, groups them by source IP, and flags any source whose alerts span two or more distinct segments within a configurable time window — the mechanism introduced in Section 2 and detailed further in Section 5.

3.4 Live Dashboard

A browser-based dashboard, served by the same Flask application, polls the API every two seconds and displays a live segment topology view, a raw alert feed, and the correlation engine's findings. It is covered in Section 7.

4. Detection Techniques

Each sensor agent runs the same set of independent, threshold-based detectors, summarized in Table 1. They were deliberately kept simple and explainable rather than opaque, which suits both a lab context and the broader goal of making detection logic auditable — a design choice consistent with the lightweight, rule-driven philosophy of Snort [4].

Detector	Trigger Condition
Port Scan Detection	A source IP contacts a configurable number of distinct destination ports within a short time window.
SYN Flood Detection	A source IP sends an abnormal number of SYN packets to a single destination within a short time window.

Detector	Trigger Condition
ARP Spoof Detection	A single IP address is observed resolving to more than one MAC address.
Suspicious Payload Matching	A packet payload contains a known malicious signature keyword (e.g., SQL-injection patterns, suspicious shell commands).

Table 1. Detection techniques implemented by each sensor agent.

All thresholds (distinct-port count, SYN count, time windows, and the signature keyword list) are configurable in a single configuration file rather than hard-coded, so the sensitivity of each detector can be tuned without touching code.

5. Cross-Segment Correlation

Section 2 explained why correlation across segments, rather than per-sensor alerting alone, is what makes this system genuinely distributed in the sense described by Treaster and by Zhou et al. [2], [3]. Concretely, the correlation engine performs the following steps on every pass:

- Retrieve all alerts received within the trailing correlation window (configurable; 120 seconds by default).
- Group those alerts by source IP address.
- For any source IP whose alerts span two or more distinct segments, generate a correlated finding recording the source IP, the segments involved, the alert types involved, and the highest severity among them.

- Persist the finding so it is not re-generated on the next pass, and surface it on the dashboard.

An example finding generated during testing (Section 9) illustrates the output:

```
Source 10.13.37.66 triggered PORT_SCAN, SYN_FLOOD across 3 segments (DMZ, INTERNAL, OT_IOT) within 120s -- possible reconnaissance / lateral movement.
```

6. Implementation

6.1 Technology Stack

- Language: Python 3
- Sensor agents: pure Python detection logic; optional scapy for live packet capture
- Collector API and dashboard server: Flask
- Storage: SQLite
- Configuration: a single YAML file covering segments, detection thresholds, and correlation parameters
- Dashboard front end: HTML, CSS, and vanilla JavaScript polling the collector's JSON API

6.2 Project Structure

```
distributed_ids/
|-- agent/
|   |-- sensor_agent.py      # per-segment sensor (simulate or live capture)
|   +-- detections.py        # detection rule engine
```

```
|-- collector/
|   |-- app.py                    # Flask API + dashboard server
|   |-- correlation.py            # cross-segment correlation engine
|   +-- storage.py                # SQLite persistence layer
|-- dashboard/
|   +-- templates/index.html      # live dashboard UI
|-- simulate_multi_segment.py     # launches all segment agents at once
|-- run.py                        # one-command launcher (no Docker needed)
|-- Dockerfile / docker-compose.yml
|-- config.yaml
+-- requirements.txt
```

6.3 Design Decisions

- Alerts are normalized into a common event schema before detection, so the same detection engine runs unmodified whether the input is simulated or captured live — no branching logic between the two modes.
- Detectors are independent, single-responsibility classes; adding a new detection technique means adding one class and registering it, without touching the sensor agent or collector.
- Correlation runs as a separate background process/thread rather than inline with alert ingestion, so a slow correlation pass cannot block incoming alerts.

7. Live Dashboard

The dashboard gives a real-time view of the whole system: a segment topology strip showing each monitored segment and how many alerts it has generated, a live alert feed, and a cross-segment findings panel showing correlation engine output. The interface polls the collector's API every two seconds.

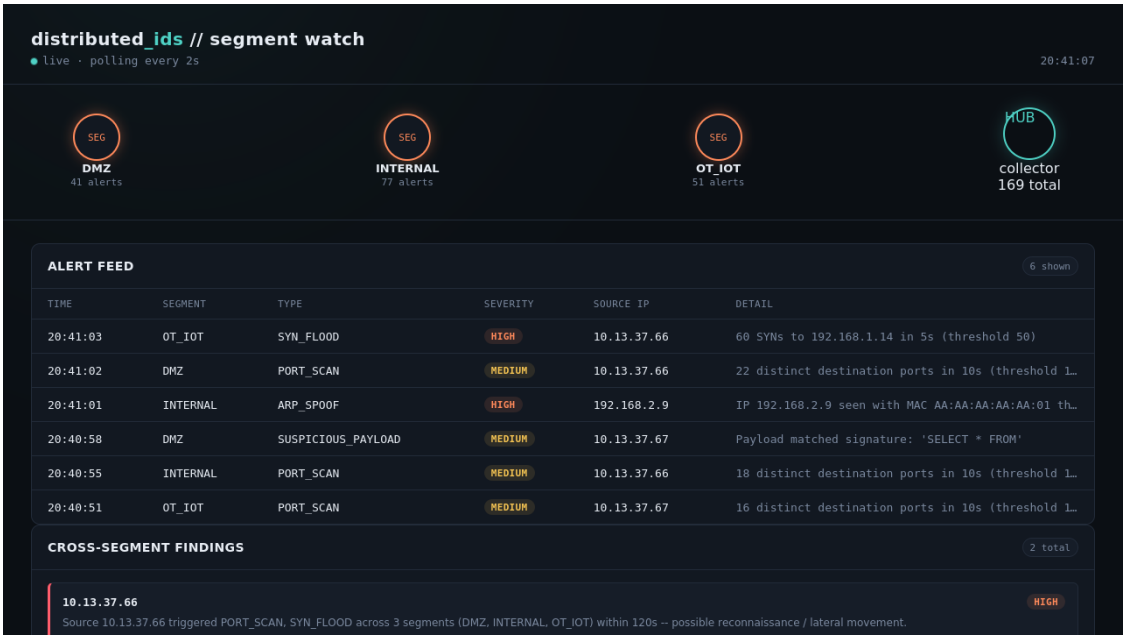

Figure 2. The live dashboard during a test run, showing segment topology, the alert feed, and a cross-segment finding.

8. Running the Framework

The system was built to be easy to demonstrate. The primary path requires no Docker, no administrator/root privileges, and no special network configuration:

```
python run.py
```

This single command starts the collector, waits for it to become available, launches the simulated sensor agents for all configured segments, and opens the dashboard in a browser automatically. It shuts down cleanly on Ctrl+C.

Two alternative modes are also supported:

- Docker: "docker compose up --build" starts the collector and simulated sensors as two containers on a private network, for environments where Docker with a working virtualization backend is available.
- Live capture: running a sensor agent with "--mode live" against a real network interface or mirror port captures genuine traffic via scapy, using the same detection engine as simulation mode.

9. Testing and Results

The complete pipeline — sensor agents, the collector API, SQLite persistence, and the correlation engine — was tested end-to-end using the simulation mode described above. Three simulated segments (DMZ, INTERNAL, and OT_IOT) were run simultaneously against a live collector instance.

Over one test session, the system recorded the results summarized in Table 2.

Metric	Result
Total alerts recorded	145
Port scan alerts	123

Metric	Result
SYN flood alerts	22
Cross-segment correlated findings	3
Segments monitored simultaneously	3 (DMZ, INTERNAL, OT_IOT)

Table 2. Summary results from a single end-to-end test session.

All three correlated findings correctly identified a single simulated attacker source IP whose alerts appeared across multiple segments within the correlation window, confirming that the correlation engine performs as designed. The dashboard was verified to reflect alerts and findings within its two-second polling interval, and the one-command launcher (run.py) was verified to start, run, and shut down cleanly with no orphaned processes.

10. Limitations and Future Work

As a lab-scale implementation, several simplifications were made deliberately, and are noted here rather than left implicit:

- Alert transport uses plain HTTP with a shared API key rather than mutual TLS; a production deployment would require encrypted, mutually authenticated transport between sensors and the collector.
- SQLite is sufficient for a handful of segments and a demonstration workload, but a production deployment at scale would benefit from a message queue such as Kafka between sensors and the collector, and a time-series-oriented data store.

- Detection is entirely threshold-based and rule-driven, which is explainable and tunable but will produce false positives on legitimate bursty traffic; incorporating machine-learning-based anomaly detection alongside the existing rule-based detectors is a natural next step, consistent with broader trends in the IDS literature toward hybrid rule-based and learning-based detection [6], [7].
- The correlation engine currently correlates purely on shared source IP; incorporating additional signals (e.g., destination overlap, timing patterns) could reduce false correlation on shared NAT addresses.

11. Conclusion

A single sensor watching an entire network misses attacks that move between segments, because no single vantage point ever sees more than one part of the attack. This project demonstrates that placing a lightweight sensor on every segment and correlating what those sensors independently observe closes that gap: attacker behavior that spans segments — such as reconnaissance in one zone followed by activity in another — becomes visible specifically because the system is distributed, not despite it.

The resulting framework is fully functional, tested end-to-end, and runnable with a single command, while remaining transparent about the simplifications made for a lab context and the concrete steps that would be needed to take it further toward a production deployment.

References

[1] Snapp, S. R., Brentano, J., Dias, G. V., Goan, T. L., Heberlein, L. T., Ho, C. L., Levitt, K. N., Mukherjee, B., Smaha, S. E., Grance, T., Teal, D. M., & Mansur, D. (1991). DIDS (Distributed Intrusion Detection System) — motivation, architecture, and an early prototype. In Proceedings of the 14th National Computer Security Conference (pp. 167–176).

[2] Treaster, M. (2004). A survey of distributed intrusion detection approaches. National Center for Supercomputing Applications, University of Illinois. arXiv:cs/0501001.

[3] Zhou, C. V., Leckie, C., & Karunasekera, S. (2010). A survey of coordinated attacks and collaborative intrusion detection. Computers & Security, 29(1), 124–140.

[4] Roesch, M. (1999). Snort — Lightweight intrusion detection for networks. In Proceedings of the 13th USENIX Conference on System Administration (LISA '99) (pp. 229–238). USENIX Association.

[5] Paxson, V. (1999). Bro: A system for detecting network intruders in real time. Computer Networks, 31(23–24), 2435–2463.

[6] Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Transactions on Emerging Telecommunications Technologies, 32(1), e4150.

[7] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: Techniques, datasets and challenges. Cybersecurity, 2, Article 20.

[8] Fung, C. J., & Boutaba, R. (2014). Intrusion detection networks: A key to collaborative security. CRC Press.